



Microservice Architectural Style

A.A. 2025/2026

Service Oriented Software Engineering

Gianluca Filippone

gianluca.filippone@gssi.it

Lecture(s) roadmap

- › History: from monoliths to microservices
- › Microservice characteristics
- › Pro & cons
- › Designing microservices
- › Integrating microservices
- › Deploying microservices

- › Hands on tutorial

Monoliths

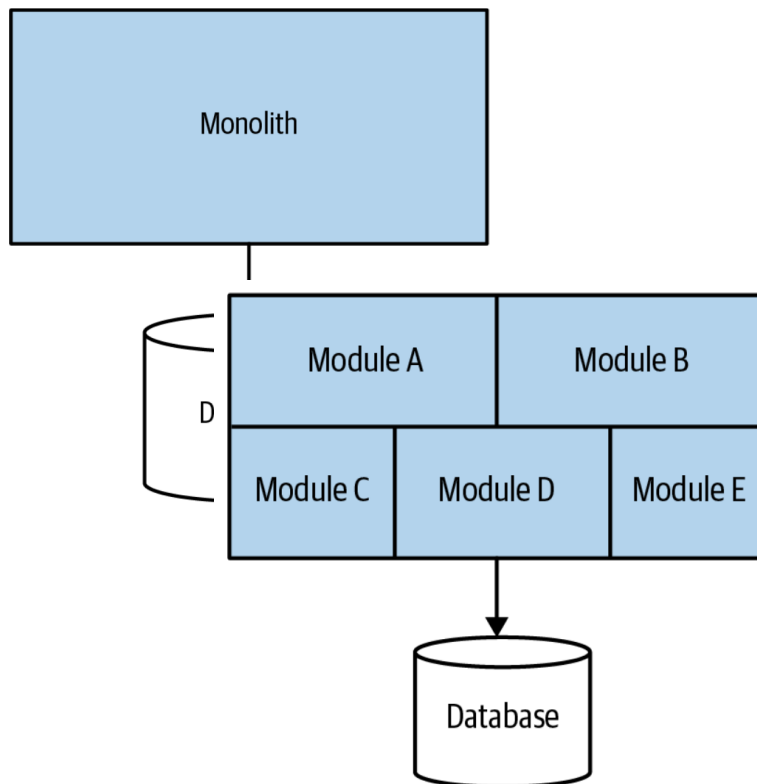
Monolith: "Large, single block of stone"

- › From greek: monos ("single") and lithos ("stone")

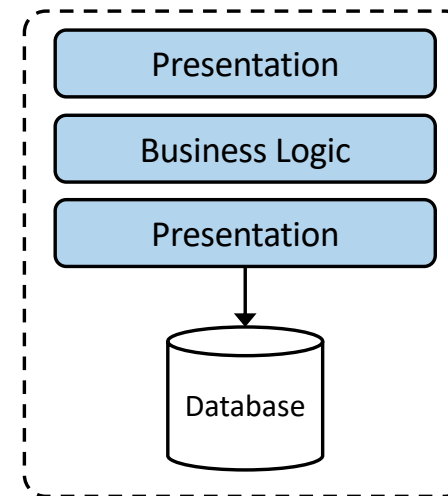


Monolithic system

Application created and deployed as a single unit



- › Implements all the system's functionalities
- › All the code is packaged in a single executable artifact
 - › E.g., a server "running" a single .war file
 - › Modules can not be executed independently
- › Traditional 2- or 3-tier applications

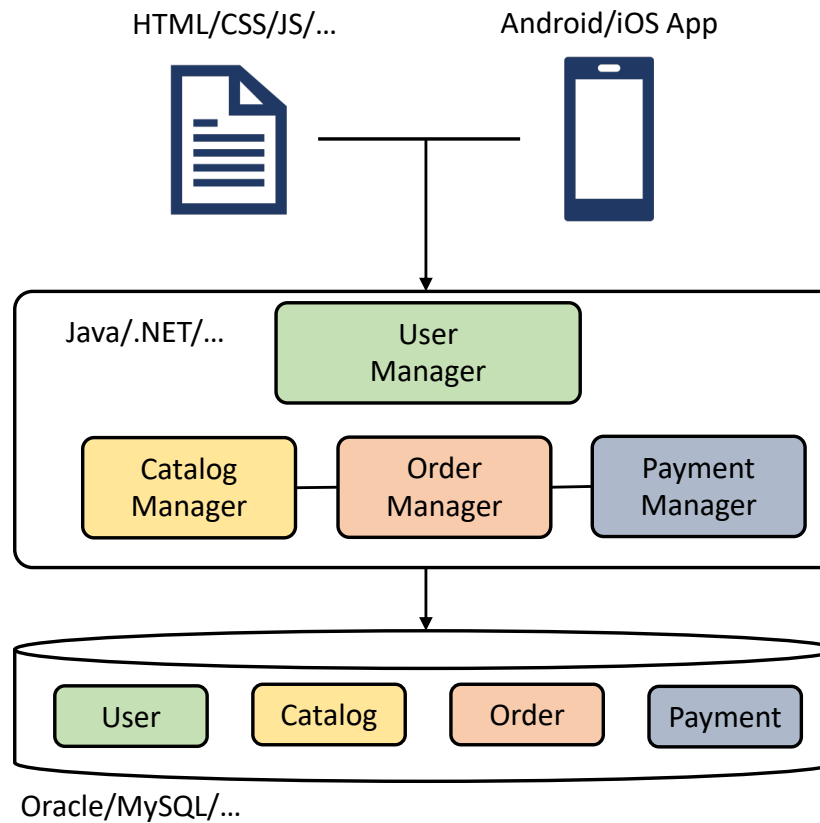


How monoliths are developed (an example)

Organization



Architecture



Deployment



Pros & cons

Benefits

- › Simple deployment technology
- › Simple monitoring
 - › A single process is running
 - › Easy to find where issues are
- › Simple code reuse
 - › To share functionalities within a system
- › Fast time-to-market
- › Note: monolithic $\neq$ legacy
 - › Legacy systems are usually monoliths, not vice-versa

Drawbacks

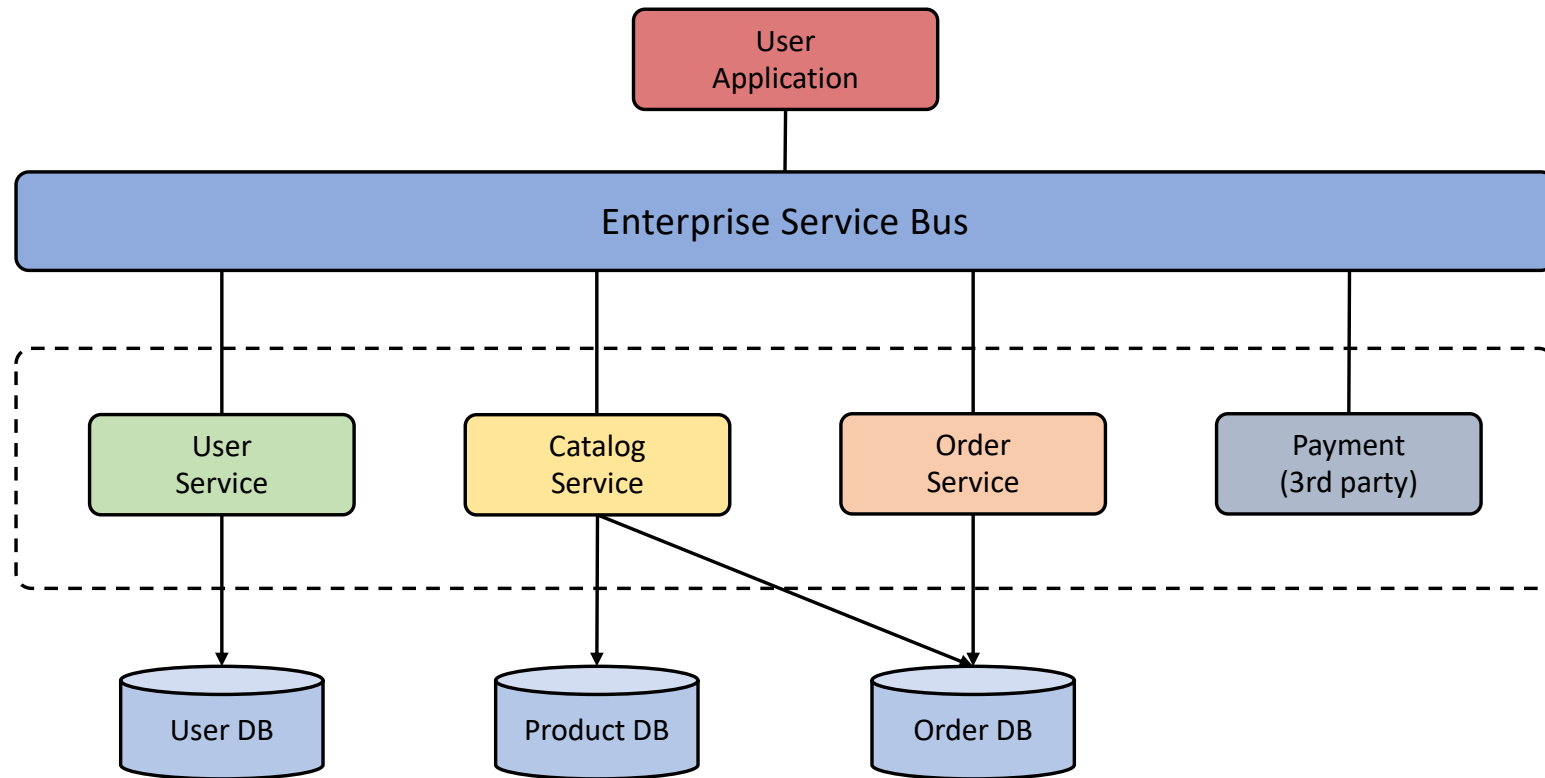
- › High coupled
 - › Everyone needs to know everything
- › Large size
 - › Difficult to maintain
- › Difficult to evolve
 - › Need to be restarted for any update
- › Sub-optimal deployment
- › Limited scalability
- › Technological lock-in

Moving towards SOA

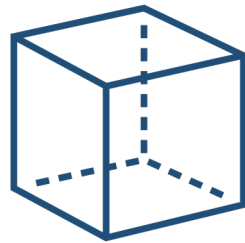
Service-oriented architectures

- › Systems made by composing reusable components
 - › Accessible through the network
 - › Using a shared communication protocol
 - › Distributed systems
 - › No single-executable artifact
 - › Loosely coupled services
 - › Compose and integrate multiple applications
 - › Including third-party and black-box
 - › Focus on the *enterprise level*
 - › Coarse-grained services
 - › No clearly-defined boundaries
- 

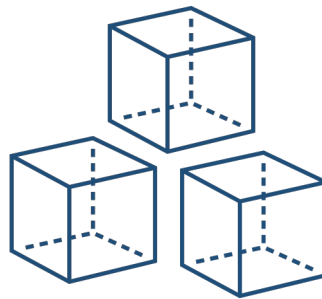
A SOA architecture



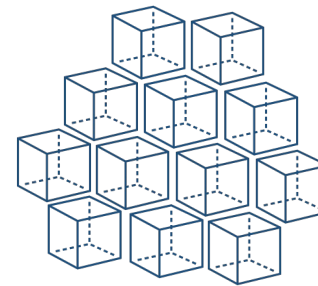
Moving towards microservices



Monolith
(Single unit)



SOA
(Coarse-grained)



Microservices
(Fine-grained)



1990

2005

2015

Microservices

"SOA done right" [Sam Newman]

*"Approach to developing a single application as a suite of **small services**, each running in its **own process** and communicating with **lightweight** mechanisms, often an HTTP resource API." [Martin Fowler]*

*"Microservices are **independently deployable** services modeled around a **business domain**. [...] A microservice architecture is based on multiple collaborating microservices. " [Sam Newman]*

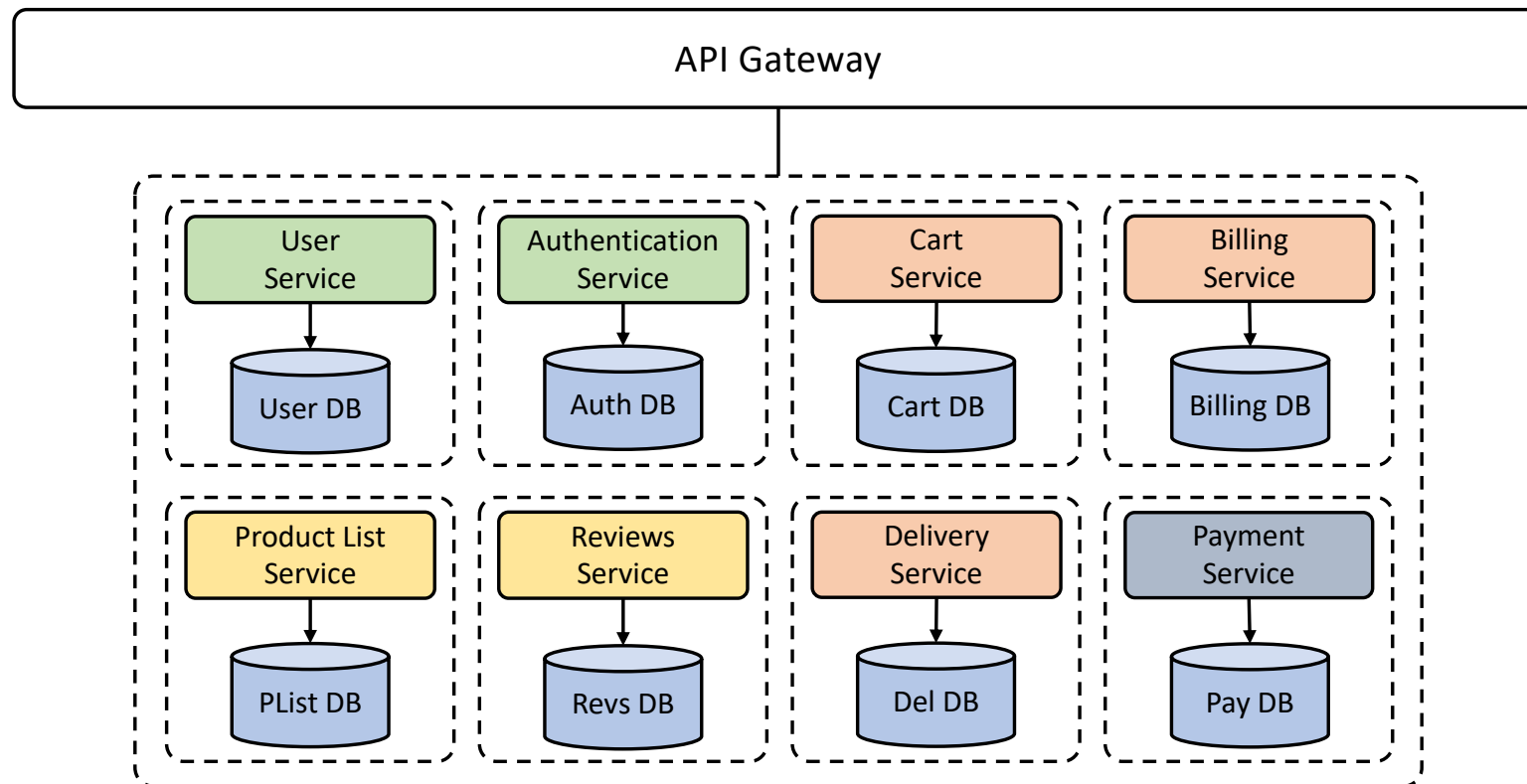
SOA

Small Services

Independent

Lightweight

A Microservice architecture



Architectural principles

- › Bounded Context
 - › Related functionalities grouped into the same business capability
- › Single Responsibility
 - › *"Do one thing, and do it well"*
- › Lightweight
 - › Maintained and developed by a small-sized team
 - › "Two pizza rule"
- › Independency
 - › Operationally independent from each other
 - › Communication only through interfaces

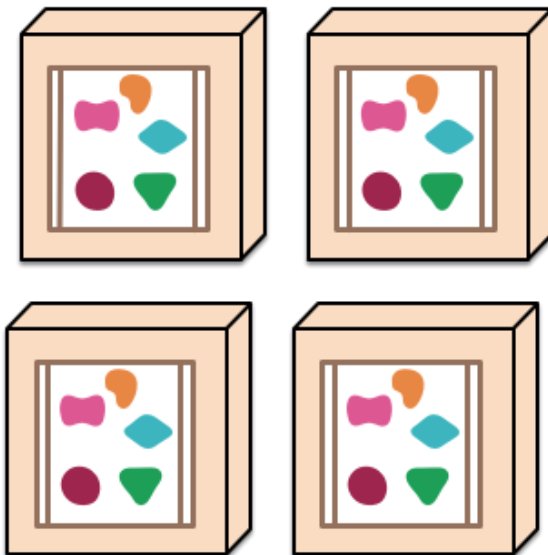
High cohesive
Loose coupled

Monoliths vs Microservices

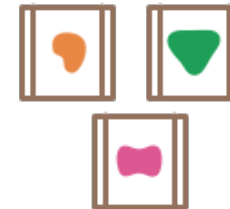
A monolithic application puts all its functionality into a single process...



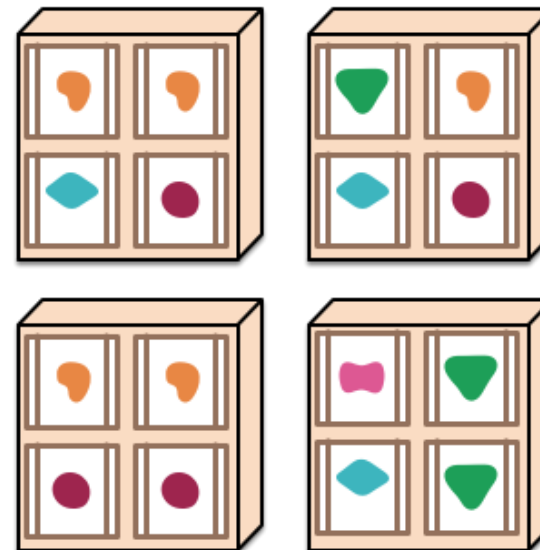
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



Benefits of microservices

- › Scalability
 - › Replicate instances of loaded microservices
 - › Better resource management
- › Robustness
 - › No single-points of failure
 - › Enhance availability
- › Maintainability and ease of development
 - › Functionalities can be upgraded independently
 - › Continuous delivery & Continuous integration
- › Flexibility
 - › No technological constraint

Monoliths vs Microservices (again)

Monoliths issues

- › Difficult to maintain
- › Difficult to evolve
- › Sub-optimal deployment
- › Limited scalability
- › Technological lock-in

Microservice benefits

- › Easy to maintain
 - › Small sized
- › Enable Continuous integration
 - › Changes do not require reboot
- › Containerization
 - › Computational effort is distributed
- › Enhanced scalability
 - › Replication of microservice
- › Flexibility
 - › Each microservice owns its technological stack

Microservices are not a silver bullet

As for any architectural style, there are drawbacks...

- › Complexity
 - › Distribution
 - › Deployment
- › Performances
 - › Network communication introduces latency
- › Difficult management
 - › Skills required
- › Difficult identification
 - › Risky process
- › Higher time-to-market
- › Sometimes is better to realize a monolith
 - › Startup companies

Designing microservices

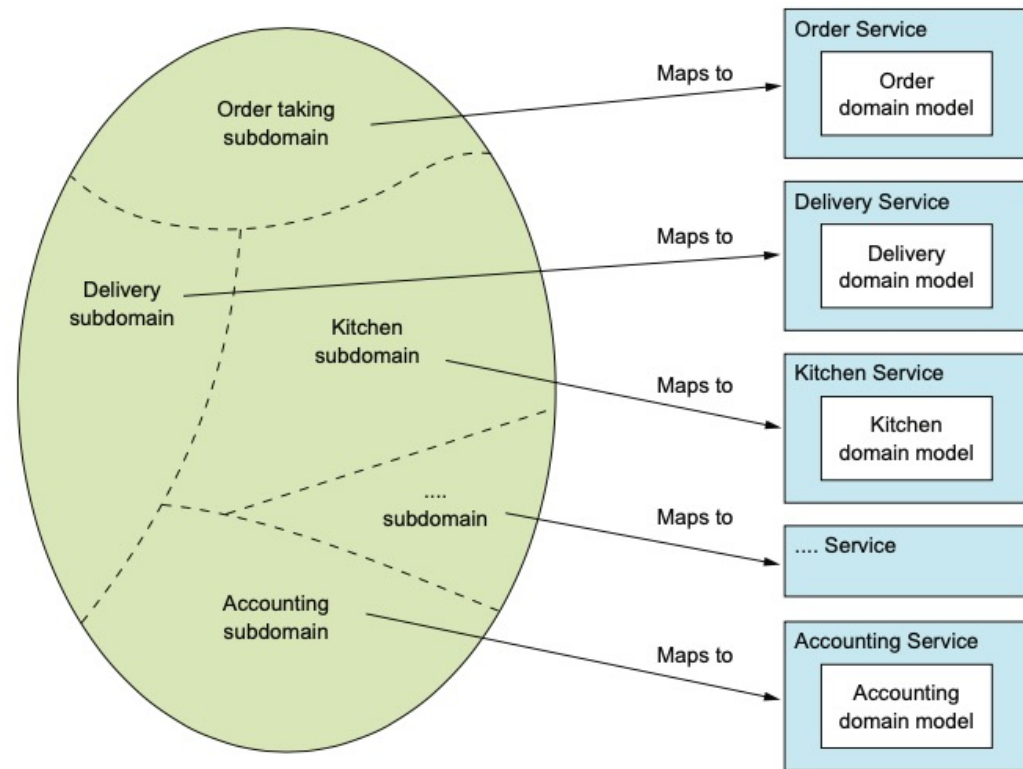
- › Follow bounded context principles
 - › Identify capabilities
 - › Define clear boundaries
- › Ensure high cohesion
 - › "Code that changes together, stays together"
 - › Related capabilities are likely to be very cohesive
- › Ensure loose coupling
 - › "The more things are coupled, the more they will change together"
 - › Reduce code and domain interdependencies
 - › Microservices should know as little as possible about others (APIs only)
 - › Microservices should not be *chatty*

Domain-driven design of microservices

Domain-driven design: build applications by focusing on an object-oriented domain model

› The domain model captures knowledge about the system

- › Identify aggregates into the domain model
- › An aggregate is a cluster of domain objects
- › The scope of each (sub)domain model is a *bounded context*
- › Map aggregates and bounded contexts to microservices



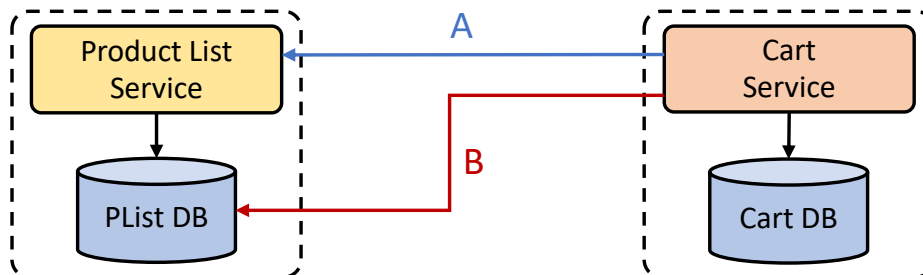
Integrating microservices

Problem: microservices need to communicate

› *Microservices work well if they are integrated well*

› Principles

- › Avoid breaking changes
- › APIs should be technologically agnostic



If using a shared DB (**B**):

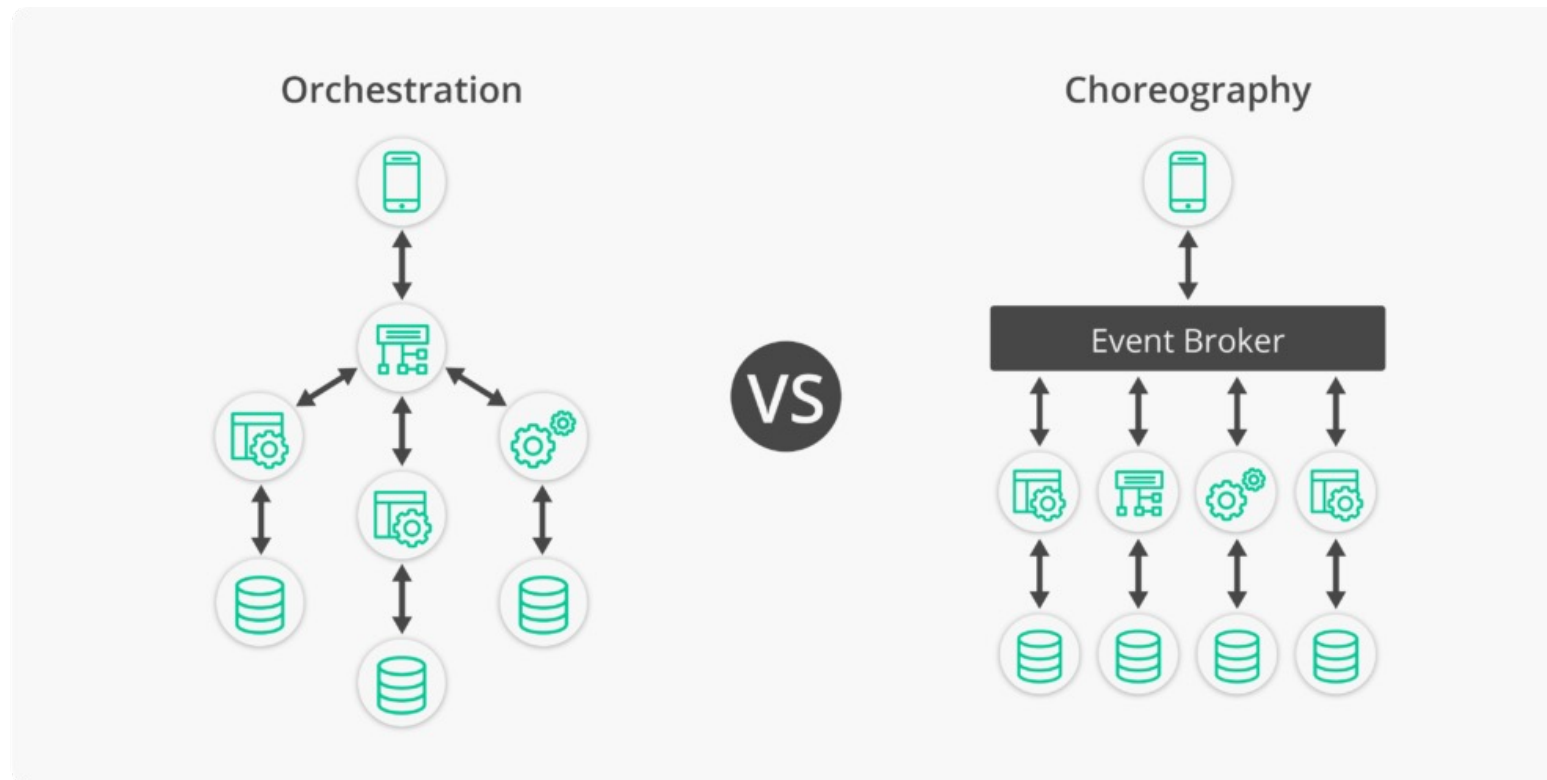
- › A change in Plist DB would affect both MSs
- › Technological constraint

- › Avoid resource sharing
- › Always prefer communication (**A**)

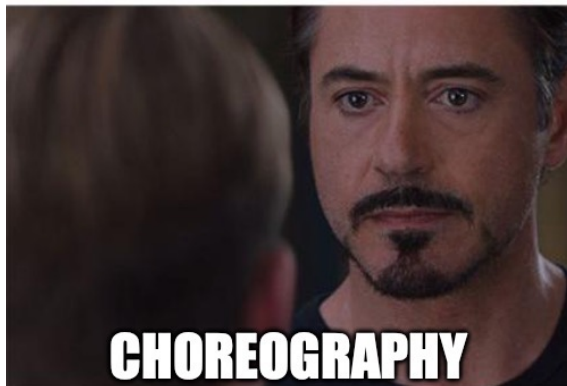
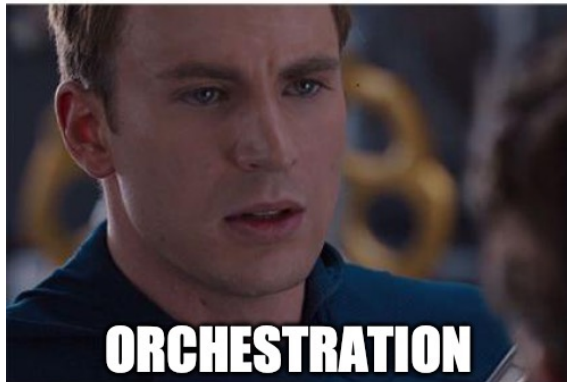
Microservice communication

- › Use APIs to hide the internal structure of the microservices
 - › Reduces coupling
 - › Avoid the dependency of a particular technology
- › **Synchronous** communication
 - › Request-response interaction through APIs
 - › Usually REST protocol
 - › May affect performances
- › **Asynchronous** communication
 - › Request-response (the consumer service does not wait for the response)
 - › Publish-subscribe
 - › Good for long-running tasks
 - › Usually more difficult to implement

Microservice composition



Microservice composition (2)



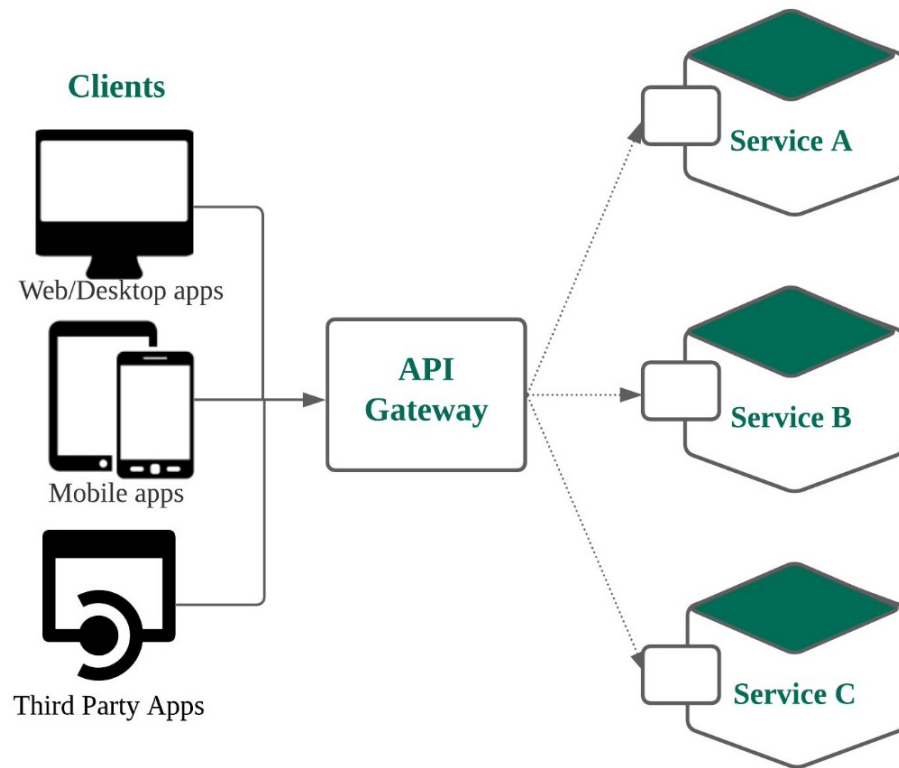
› Orchestration

- › A central coordinator controls the interaction
- › Simple
- › Easy to manage
- › Tight coupling
- › Single-point of failure
- › ...may lead to a "distributed monolith"

› Choreography

- › The control is distributed
- › Everyone knows what to do (independence)
- › Loose coupling
- › Decentralized
- › Complex

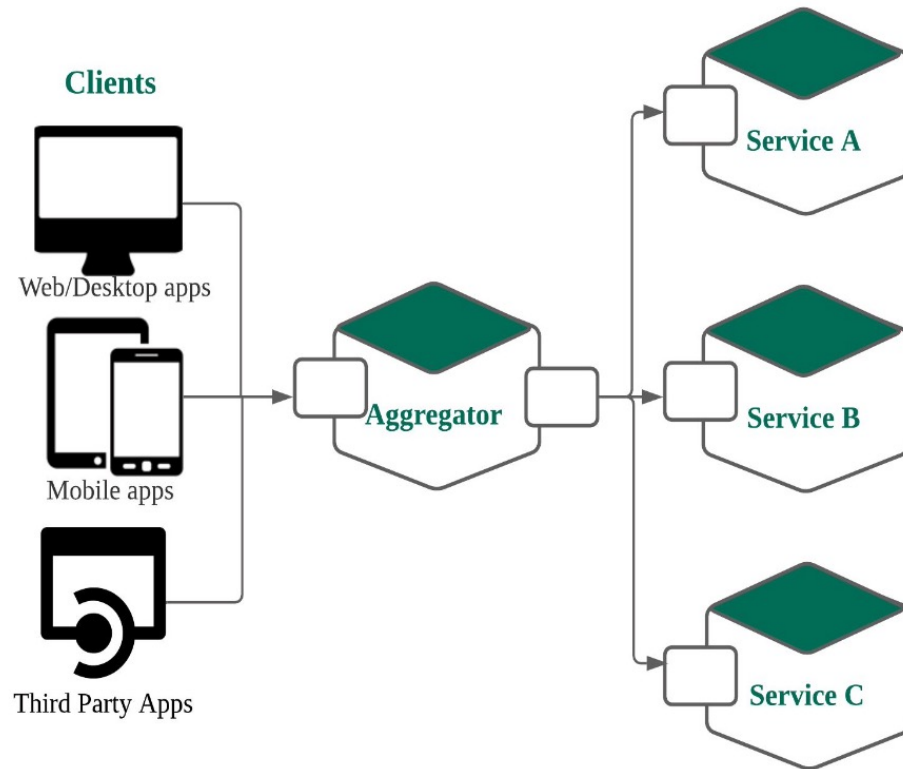
API Gateway pattern



An **API Gateway** routes requests towards the microservices

- › Single point of entry
- › Proxies communication
- › Authentication can be provided by the API Gateway
 - › "Inner" communication can be trusted
- › Adapts the communication protocols

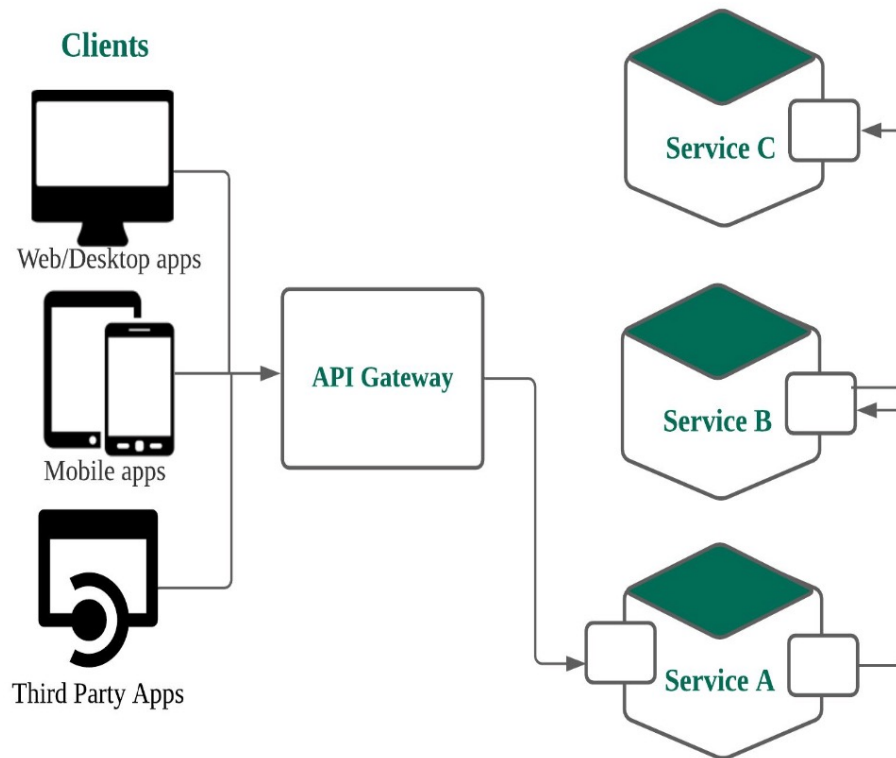
Aggregator pattern



An **Aggregator** microservice composes responses from microservices

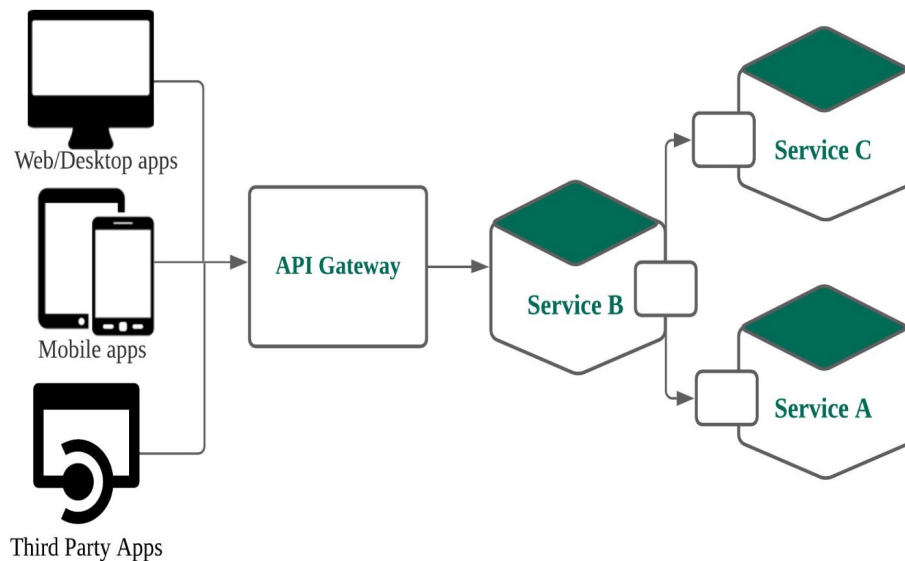
- › Similar to the API Gateway
 - › ...but convenient if aggregation is complex and part of the system business logic
- › Can act as orchestrator

Chained microservices



- › Microservices can be **chained** if they have some interdependency
- › More similar to choreographies
- › Be careful: introduces coupling
 - › Use only if their role is clearly defined

Branch microservice



- › Composition of aggregator and chain pattern
- › Useful to parallelize operations
 - › Allows to obtain data from different microservices

Deploying microservices

Containerization is a flexible way to gain independency and isolation

